Variables and Datatypes in JS

05 April 2026 18:05

Variables in JavaScript

A variable is like a container that stores data.

```
> let name = "Deeptanarayan";
undefined
> name
'Deeptanarayan'
> let age = 19;
undefined
> age
19
```

- "name" stores a text value.
- "age" stores a number

Javascript provides three ways to declare variables:

A. var
B. let
C. const

A. var: is the old way of declaring variables.

```
> var city="Kolkata";
undefined
> console.log(city)
Kolkata
undefined
```

```
> if (true){
| var y = 100;
| }
undefined
> console.log(y)
100
undefined
> y
100
```

If we declare the variable by using var keyword. Then its access is available outside the block.

B. let: Is the modern way to declare variables.

```
> let fname = 'Harry';
undefined
> console.log
[Function: log]
> console.log(fname);
Harry
undefined
> let x = 100;
undefined
```

```
> if(true){
| let m = 100;
| }
undefined
> console.log(m)
Uncaught ReferenceError: m is not defined
```

```
> if(true){
| let m = 100;
| console.log(m);
| }
100
undefined
```

let

Vs

```
> if (true){
| var y = 100;
| }
undefined
> console.log(y)
100
undefined
> y
100
```

var

C. **const:** Is used when the value should not change.

```
> const country = 'India';
undefined
> country = 'USA';
Uncaught TypeError: Assignment to constant variable.
> const pi = 3.14;
undefined
> console.log(pi);
3.14
undefined
```

Data Types in JavaScript

Javascript data types are divided into two types:

1. Primitive type
2. Non-Primitive(Reference) Data Types

Primitive Data Types

Primitive types store a single value.

There are 7 Primitive types in Javascript:

1. String
2. Number
3. Boolean
4. Undefined
5. Null
6. BigInt
7. Symbol

1. **String**

```
> let studentName = "Deeptanarayan"
undefined
> let message = 'Hello world!'
undefined
> let greeting = `Welcome`
undefined
> console.log(greeting)
Welcome
undefined
```

Strings can use single quotes, double quotes or backticks.

Usage of backticks:

```
> console.log(`Sum = ${m + p}`)
Sum = 15
undefined
> console.log("Sum = ${m + p}")
Sum = ${m + p}
undefined
> console.log('Sum = ${m + p}')
Sum = ${m + p}
undefined
```

Backticks are "Extended functionality" quote. Embed variables and expressions into

a string by wrapping them in ${…}.

Double and single quotes are simple quotes. There's practically no difference between them in JavaScript.

2. Number

```
> let n = 123;
undefined
> n
123
> n = 12.345
12.345
>
```

The number type represents both integer and floating point numbers.

There are many operate. Chance for number example multiplication *, division /, addition +, subtraction -, modulus %, exponentiation **.

Besides regular numbers. There are so called. Special numeric values. Which also belongs to this data type: Infinity, -Infinity and NaN.

Infinity: It represents the mathematical Infinity. It is a special value that is greater than any number.

We can get it as a. Result of division by zero.

```
> n/0
Infinity
```

Or just reference it directly:

```
> console.log(Infinity)
Infinity
undefined
> n = Infinity
Infinity
> n
Infinity

> n = -Infinity
-Infinity
> n
-Infinity
```

NaN: Represents a computational error. It is a result of an incorrect or an undefined mathematical operation, for instance:

```
> result = "Ten" / 2;
NaN
```

Such division is erroneous.

```
> result = NaN + 1
NaN
> result = NaN * 3
NaN
> result = Infinity * 3
Infinity
> result = Infinity + 1
Infinity
```

NaN and Infinity are sticky. Any further mathematical operation on NaN returns NaN And Infinity returns Infinity.

```
> result = NaN ** 0;
1
> result = Infinity ** 0;
1
```

BigInt

In Javascript "number" type cannot safely represent integer values larger than 2^{53} - 1 (i.e., 9007199254740991) or less than -(2^{53} - 1) for negatives.

To be really precise. Number type can store large integers (up to 1.7976931348623157 * 10^{308}), But outside of the safe integer range **+/-(2^{53} - 1)** There will be a. Season error because not all the digits fit into the fixed 64 bit storage. So an approximate value may be stored.

For example, these two numbers (right above the safe range) are the same.

```
> (2**53 - 1)+1
9007199254740992
> (2**53 - 1)+2
9007199254740992
```

So to say, all odd integers greater than (2^{53} - 1) Can't be stored at all in the "number" type.

For most purpose **+/-(2^{53} - 1)** Range is quite enough, but sometimes we need the entire range of really big integers, example for cryptography or microsecond precision timestamps.

BigInt type was recently added to the language to represent integers of arbitrary length.

A **BigInt** value is created by appending 'n' to the end of an integer:

```
> const bi = 23456789763398089765345675434546789765n;
```

```
undefined
> bi
2345678976339808976534567543454678976 5n
> bi + 5n
23456789763398089765345675434546789770n
> bi = bi + 5n
Uncaught TypeError: Assignment to constant variable.
> let ci = bi + 5n
undefined
> ci
23456789763398089765345675434546789770n
>
```

As BigInt numbers are rarely needed, whenever you need it study it.

Boolean(logical type)
The boolean type. Has only two values: true and false.
This type is. Commonly used to store yes or no values: 'true' means "yes, correct" and 'false' means "no, incorrect".

```
> let nameFieldChecked = true;
undefined
> let ageFieldChecked = false;
undefined
> let isGreater = 4 > 1;
undefined
> console.log(isGreater);
true
undefined
> nameFieldChecked
true
> ageFieldChecked
false
```

The "null" value
The special null value does not belong to any of the types described above.
It forms a separate type of its own which contains only the null value.

```
> let age = null;
undefined
> age
null
```

In Javascript **null** is not a "reference to a non-existing object" or "a null pointer" like in some other languages. It's just a special value which represents "nothing", "empty" or "value unknown". The code above states that *age* is unknown.

The "undefined" value

The special value. Undefined also stands apart. It makes a type of its own, just like **null**.

The meaning of *undefined* is "value is not assigned".
If a variable is declared but not assigned, then the value is undefined.

```
> g
undefined
> g = 100
100
> g = undefined
undefined
> g
undefined
> g = 'Hello'
'Hello'
```

… But we don't recommended doing that normally one uses null to assign an empty or unknown value to a variable, while undefined is reserved as a default initial value for unassigned things.

Objects and Symbols

The *object* type is a special.

All other types are called primitive because their values can contain only a single thing (be it a string or a number or whatever) In contrast, objects are used to store collections of data and more complex entities.

The **symbol** type is used to create unique identifiers for objects. We have to mention it here for the sake of completeness, but also postpone the details till we know objects.

The typeof operator

The **typeof** operator returns the type of the operand. It's useful when we want to process values of different types differently, or just want to do a quick check.

```
> g = undefined;
undefined
> typeof g
'undefined'
> typeof 0
'number'
> typeof 10n
'bigint'
> typeof true
'boolean'
> typeof "Hello"
'string'
> typeof Symbol("id")
```

```
'symbol'
> typeof Math
'object'
> typeof null
'object'
```